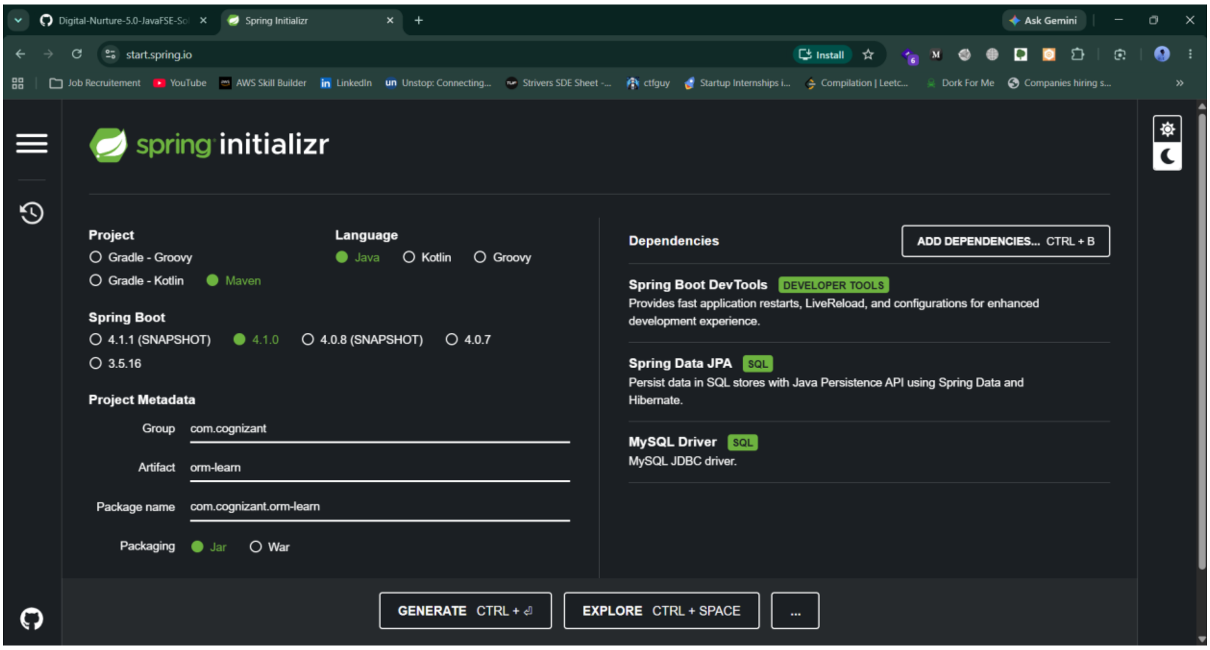
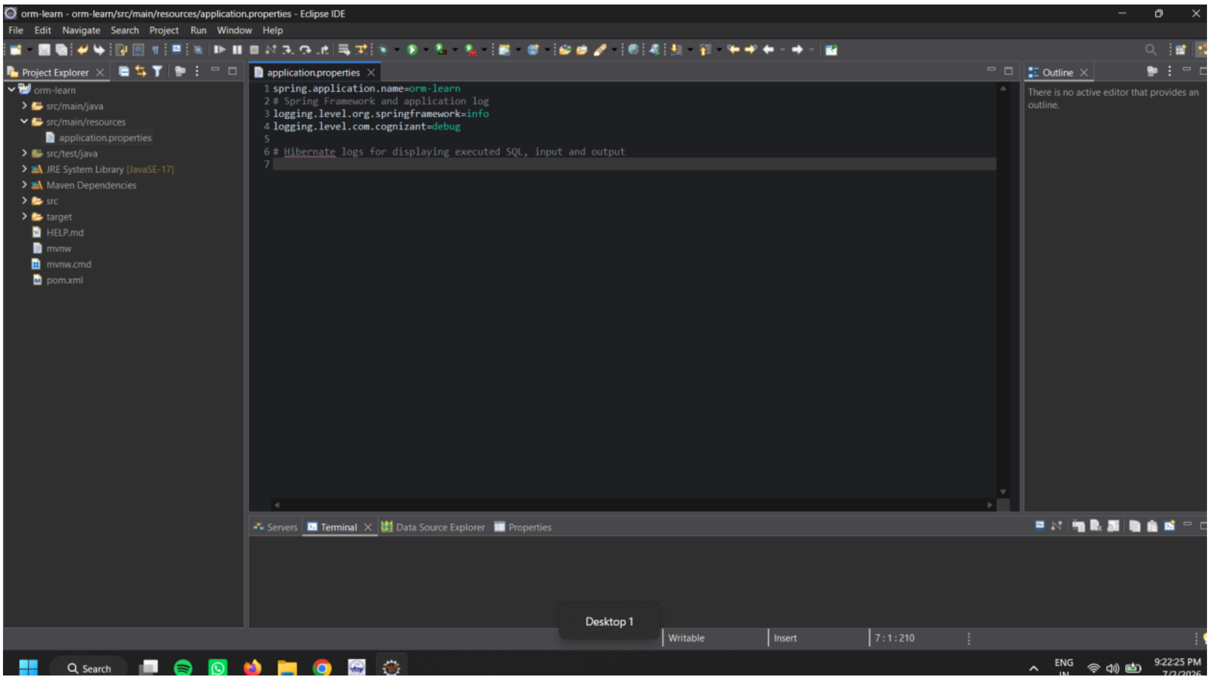
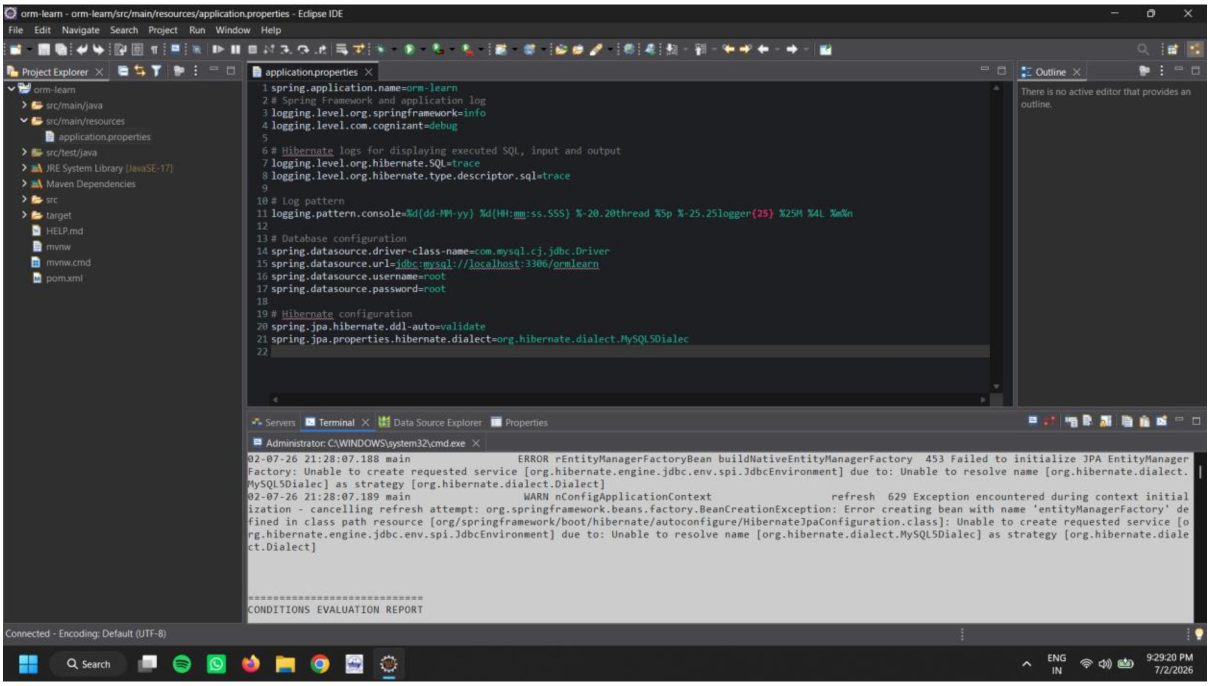
Week 2- Spring Data JPA with Hibernate - 7995206

EXERCISE 1: Spring Data JPA - Quick Example

- Go to https://start.spring.io/
- Change Group as “com.cognizant”
- Change Artifact Id as “orm-learn”
- In Options > Description enter "Demo project for Spring Data JPA and Hibernate"
- Click on menu and select "Spring Boot DevTools", "Spring Data JPA" and "MySQL Driver"
- Click Generate and download the project as zip
- Extract the zip in root folder to Eclipse Workspace
- Import the project in Eclipse "File > Import > Maven > Existing Maven Projects > Click Browse and select extracted folder > Finish"
- Create a new schema "ormlearn" in MySQL database. Execute the following commands to open MySQL client and create schema.
- In orm-learn Eclipse project, open src/main/resources/application.properties and include the below database and log configuration.
- Build the project using ‘mvn clean package -Dhttp.proxyHost=proxy.cognizant.com -Dhttp.proxyPort=6050 -Dhttps.proxyHost=proxy.cognizant.com -Dhttps.proxyPort=6050 -Dhttp.proxyUser=123456’ command in command line
- Include logs for verifying if main() method is called.

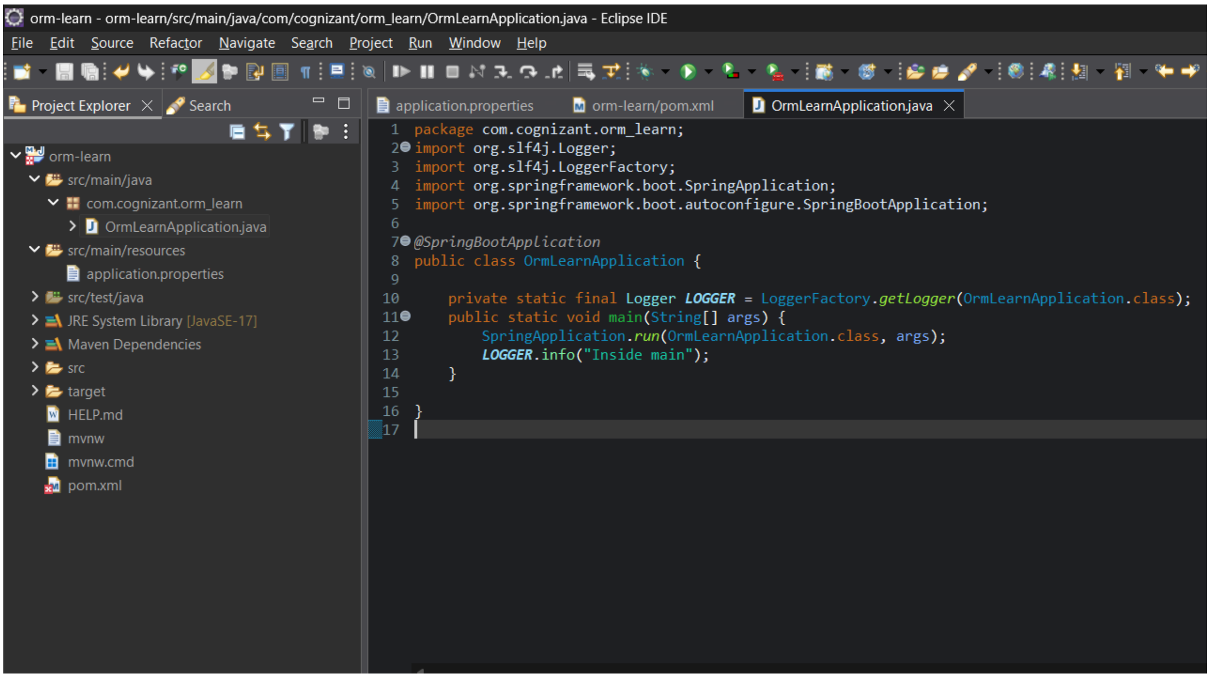


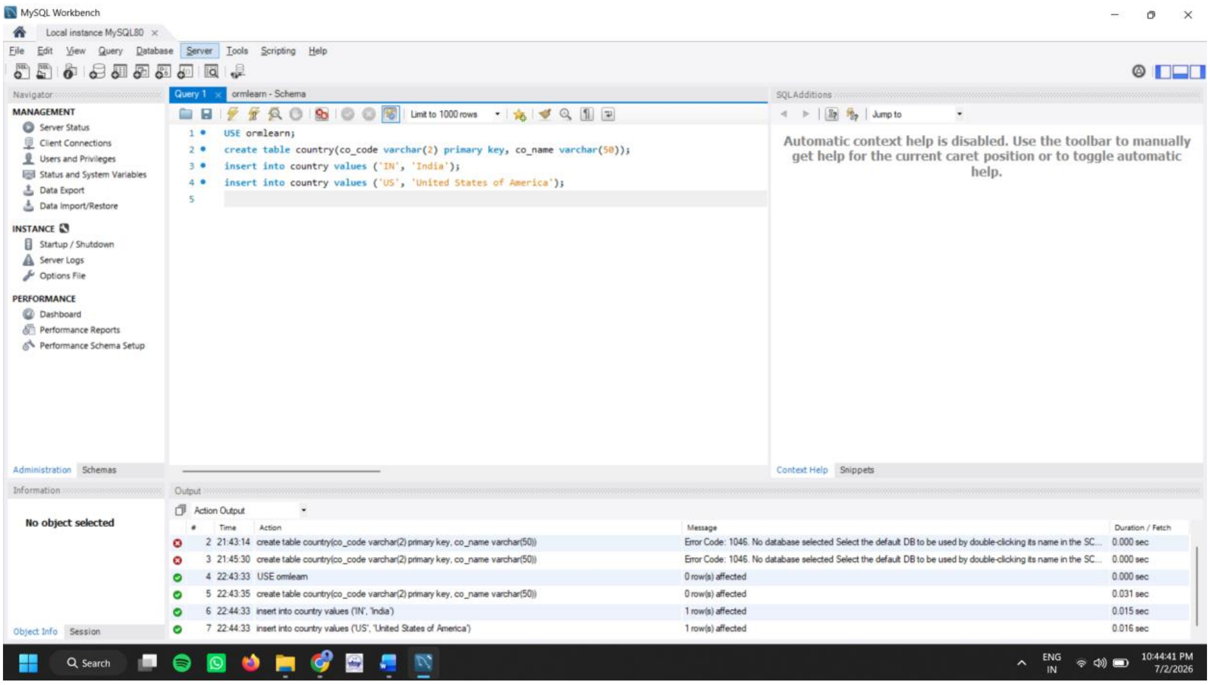


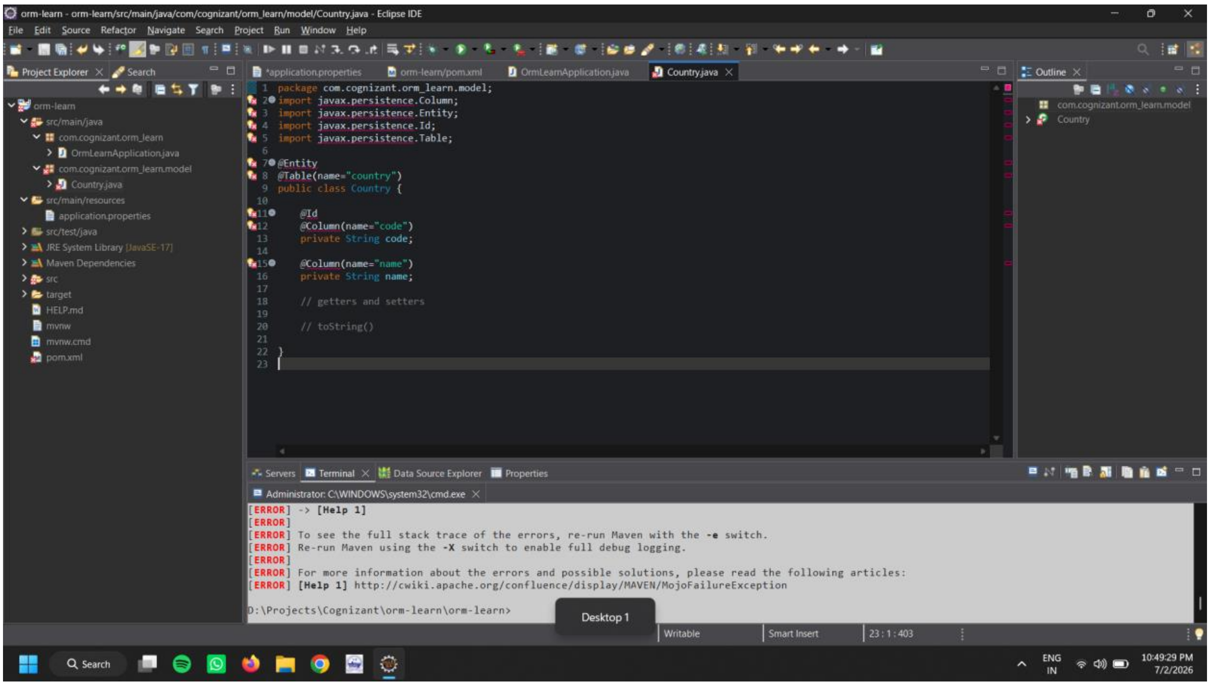


Persistence Class - com.cognizant.orm-learn.model.Country

- Open Eclipse with orm-learn project
- Create new package com.cognizant.orm-learn.model
- Create Country.java, then generate getters, setters and toString() methods.
- Include @Entity and @Table at class level
- Include @Column annotations in each getter method specifying the column name.


orm-learn - orm-learn/src/main/java/com/cognizant/orm_learn/OrmLearnApplication.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Project Explorer
Search
application.properties
orm-learn/pom.xml
OrmLearnApplication.java
orm-learn
src/main/java
com.cognizant.orm_learn
OrmLearnApplication.java
src/main/resources
application.properties
src/test/java
JRE System Library [JavaSE-17]
Maven Dependencies
src
target
HELP.md
mvnw
mvnw.cmd
pom.xml
package com.cognizant.orm_learn;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class OrmLearnApplication {
private static final Logger LOGGER = LoggerFactory.getLogger(OrmLearnApplication.class);
public static void main(String[] args) {
SpringApplication.run(OrmLearnApplication.class, args);
LOGGER.info("Inside main");
}
}


MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Performance Reports
Performance Schema Setup
Query 1
ormlearn - Schema
Limit to 1000 rows
USE ormlearn;
create table country(co_code varchar(2) primary key, co_name varchar(50));
insert into country values ('IN', 'India');
insert into country values ('US', 'United States of America');
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Administration
Schemas
Context Help
Snippets
Information
No object selected
Object Info
Session
Output
Action Output
Time Action Message Duration / Fetch
2 21:43:14 create table country(co_code varchar(2) primary key, co_name varchar(50)) Error Code: 1046. No database selected Select the default DB to be used by double-clicking its name in the SC... 0.000 sec
3 21:45:30 create table country(co_code varchar(2) primary key, co_name varchar(50)) Error Code: 1046. No database selected Select the default DB to be used by double-clicking its name in the SC... 0.000 sec
4 22:43:33 USE ormlearn 0 row(s) affected 0.000 sec
5 22:43:35 create table country(co_code varchar(2) primary key, co_name varchar(50)) 0 row(s) affected 0.031 sec
6 22:44:33 insert into country values ('IN', 'India') 1 row(s) affected 0.015 sec
7 22:44:33 insert into country values ('US', 'United States of America') 1 row(s) affected 0.016 sec
Search
ENG
IN
10:44:41 PM
7/2/2026



EXERCISE 2 Difference between JPA, Hibernate and Spring Data JPA

Java Persistence API (JPA)

- JSR 338 Specification for persisting, reading and managing data from Java objects
- Does not contain concrete implementation of the specification
- Hibernate is one of the implementation of JPA

Hibernate

- ORM Tool that implements JPA

Spring Data JPA

- Does not have JPA implementation, but reduces boiler plate code
- This is another level of abstraction over JPA implementation provider like Hibernate
- Manages transactions

Refer code snippets below on how the code compares between Hibernate and Spring Data JPA

Hibernate

/* Method to CREATE an employee in the database */

public Integer addEmployee(Employee employee){

Session session = factory.openSession();

Transaction tx = null;

Integer employeeID = null;

```
    try {
      tx = session.beginTransaction();
      employeeID = (Integer) session.save(employee);
      tx.commit();
    } catch (HibernateException e) {
      if (tx != null) tx.rollback();
      e.printStackTrace();
    } finally {
      session.close();
    }
    return employeeID;
  }
```

Spring Data JPA

EmployeeRespository.java

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {


}
```

EmployeeService.java

```
   @Autowire
   private EmployeeRepository employeeRepository;

   @Transactional
   public void addEmployee(Employee employee) {
      employeeRepository.save(employee);
   }
```

EXERCISE 3 Implement services for managing Country

An application requires for features to be implemented with regards to country. These features needs to be supported by implementing them as service using Spring Data JPA.

- Find a country based on country code
- Add new country
- Update country
- Delete country
- Find list of countries matching a partial country name

Before starting the implementation of the above features, there are few configuration and data population that needs to be incorporated. Please refer each topic below and implement the same.

Explanation for Hibernate table creation configuration

- Moreover the ddl-auto defines how hibernate behaves if a specific table or column is not present in the database.
    - create - drops existing tables data and structure, then creates new tables
    - validate - check if the table and columns exist or not, throws an exception if a matching table or column is not found
    - update - if a table does not exists, it creates a new table; if a column does not exists, it creates a new column
    - create-drop - creates the table, once all operations are completed, the table is dropped